

## 1. Key Decisions of the Solitaire Project (2 points)

|                                          |                                                                                                                 |
|------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Object-oriented programming language     | Python                                                                                                          |
| GUI library                              | CustomTkinter, Tkinter                                                                                          |
| IDE (Integrated Development Environment) | VS Code                                                                                                         |
| xUnit framework                          | unittest                                                                                                        |
| Programming style guide                  | <a href="https://google.github.io/styleguide/pyguide.html">https://google.github.io/styleguide/pyguide.html</a> |
| Project hosting site                     | github.com                                                                                                      |
| Other decisions if applicable            |                                                                                                                 |

## 2. Unit Testing

Find a tutorial on the unit test framework you have chosen and write at least two xUnit tests of a program you have written or found elsewhere. Attach here (1) the screenshot of your program execution and (2) the source code of your program.

Tutorial for Python unittest: <https://docs.python.org/3/library/unittest.html>

I have two files: `caesar_cipher.py`, and `test_caesar_cipher.py` (where the xUnit tests for `caesar_cipher.py` are located). `caesar_cipher.py` displays a menu allowing the user to perform a caesar cipher or caesar decipher, as the filename suggests.

(1) Screenshot of program execution (`test_caesar_cipher.py`):

```
PS C:\Users\13146\School\Year 1\CS101 - Python> python -m unittest -v test_caesar_cipher.py
test_decode (test_caesar_cipher.TestCaesarCipher.test_decode) ... ok
test_encode (test_caesar_cipher.TestCaesarCipher.test_encode) ... ok
test_spaces (test_caesar_cipher.TestCaesarCipher.test_spaces) ... ok
test_wraparound (test_caesar_cipher.TestCaesarCipher.test_wraparound) ... ok

-----
Ran 4 tests in 0.001s

OK
```

## (2) Source code

(a) caesar\_cipher.py:

```
def encode(string, shift):
    newerstring = string.strip()
    cryptostring = ''
    for i in newerstring:
        if i.isalpha():
            if i.isupper():
                if (ord(i) + shift) > 90:
                    cryptostring += (f'{chr((ord(i) + shift) - 90)}')
                else:
                    cryptostring += (f'{chr(ord(i) + shift)}')
            elif i.islower():
                if (ord(i) + shift) > 122:
                    cryptostring += (f'{chr((ord(i) + shift) - 26)}')
                else:
                    cryptostring += (f'{chr(ord(i) + shift)}')
            else: #To keep spaces where they are
                cryptostring += (f'{i}')

    cryptostring = cryptostring.upper()

    return cryptostring

def decode(string, shift):
    newerstring = string.strip()
    decryptstring = ''
    newerstring = newerstring.upper()
    for i in newerstring:
        if i.isalpha():
            if (ord(i) - shift) < 65:
                decryptstring += (f'{chr((ord(i) - shift) + 26)}')

            else:
                decryptstring += (f'{chr(ord(i) - shift)}') #Double check

        else: #To keep spaces where they are
```

```

        decryptstring += (f'{i}')

    return decryptstring

def main():
    while True:
        print('MAIN MENU:')
        print('1) Encode a string')
        print('2) Decode a string')
        print('Q) Quit')
        choice = input('Enter your selection: ')

        if choice == '1':
            print()
            encrypt = input('Enter (brief) text to encrypt: ')
            shiftpos = int(input('Enter the number to shift letters by:
'))

            print(f'Encrypted: {encode(encrypt, shiftpos)}')
            print()
            continue

        elif choice == '2':
            print()
            decrypt = input('Enter (brief) text to decrypt: ')
            shiftneg = int(input('Enter the number to shift letters by:
'))

            print(f'Decrypted: {decode(decrypt, shiftneg)}')
            print()
            continue

        elif choice == 'Q':
            break

        else:
            print()
            print('Error: Invalid Selection')
            print()
            continue

if __name__ == "__main__":

```

```
main()
```

(b) test\_caesar\_cipher.py:

```
import unittest
from caesar_cipher import encode, decode

class TestCaesarCipher(unittest.TestCase):

    def test_encode(self):
        self.assertEqual(encode("HELLO", 3), "KHOOR")

    def test_decode(self):
        self.assertEqual(decode("KHOOR", 3), "HELLO")

    def test_wraparound(self):
        self.assertEqual(encode("xyz", 3), "ABC")

    def test_spaces(self):
        self.assertEqual(encode("hello world", 3), "KHOOR ZRUOG")

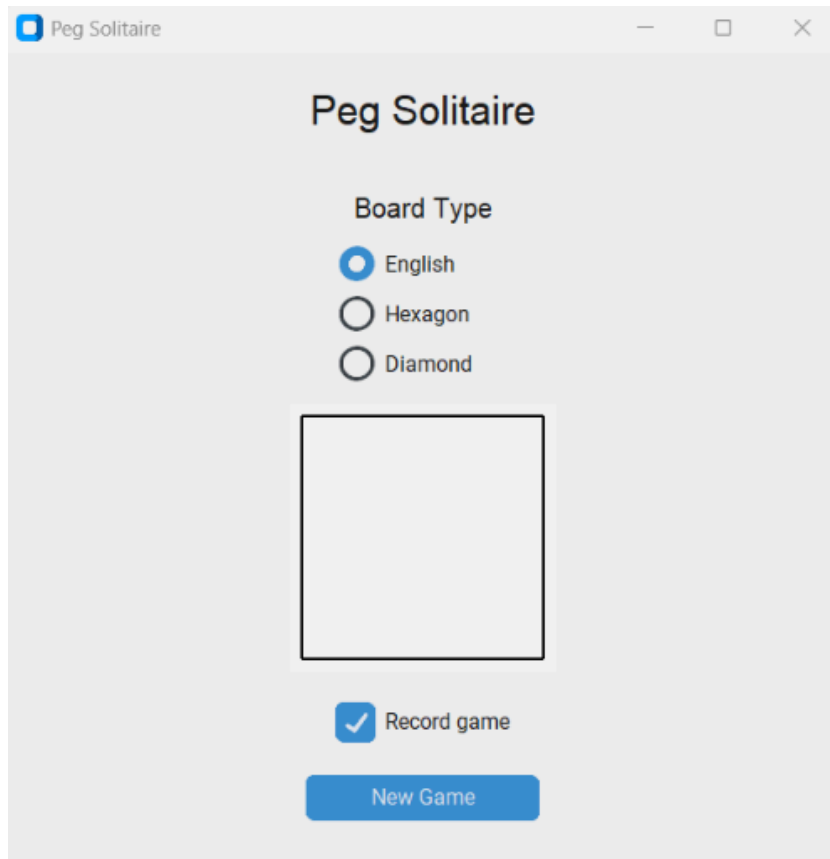
if __name__ == "__main__":
    unittest.main()
```

### 3. GUI programming

Write a GUI program in the language you have chosen for your Solitaire project. The GUI of your program must include text, lines, a check box, and radio buttons. In this assignment, any GUI program of your own work is acceptable.

Attach here (1) the screenshot of your program execution and (2) the source code of your program.

(1) Program execution screenshot:



## (2) Program source code

```
import customtkinter as ctk
import tkinter as tk

def main():
    app = ctk.CTk()
    app.title("Peg Solitaire")
    app.geometry("500x400")

    # Text
    title = ctk.CTkLabel(
        app,
        text="Peg Solitaire",
        font=("Arial", 24)
    )
```

```
title.pack(pady=20)

board_type_label = ctk.CTkLabel(
    app,
    text="Board Type",
    font=("Arial", 16)
)
board_type_label.pack(pady=(10, 5))

# Radio buttons
board_type = tk.StringVar(value="English")

english_radio = ctk.CTkRadioButton(
    app,
    text="English",
    variable=board_type,
    value="English"
)
english_radio.pack(pady=4)

hexagon_radio = ctk.CTkRadioButton(
    app,
    text="Hexagon",
    variable=board_type,
    value="Hexagon"
)
hexagon_radio.pack(pady=4)

diamond_radio = ctk.CTkRadioButton(
    app,
    text="Diamond",
    variable=board_type,
    value="Diamond"
)
diamond_radio.pack(pady=4)

# Lines
canvas = tk.Canvas(
    app,
    width=200,
```

```
        height=200,
        highlightthickness=0
    )
    canvas.pack(pady=10)

    canvas.create_line(10, 10, 190, 10, width=2)
    canvas.create_line(190, 10, 190, 190, width=2)
    canvas.create_line(190, 190, 10, 190, width=2)
    canvas.create_line(10, 190, 10, 10, width=2)

    # Checkbox
    record_checkbox = ctk.CTkCheckBox(
        app,
        text="Record game"
    )
    record_checkbox.pack(pady=10)

    # Extra button
    new_game_button = ctk.CTkButton(
        app,
        text="New Game"
    )
    new_game_button.pack(pady=10)

    app.mainloop()

if __name__ == "__main__":
    main()
```